# PilotFish HL7 MLLP (Simple) Transport – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

## At a glance (plain English)

This transport **sends an HL7 message over TCP** using the minimal MLLP framing (start block, lines with CR, end block + CR) — without validating message structure. It can wait for an ACK, optionally keep the socket open, and hand the ACK body to another listener.

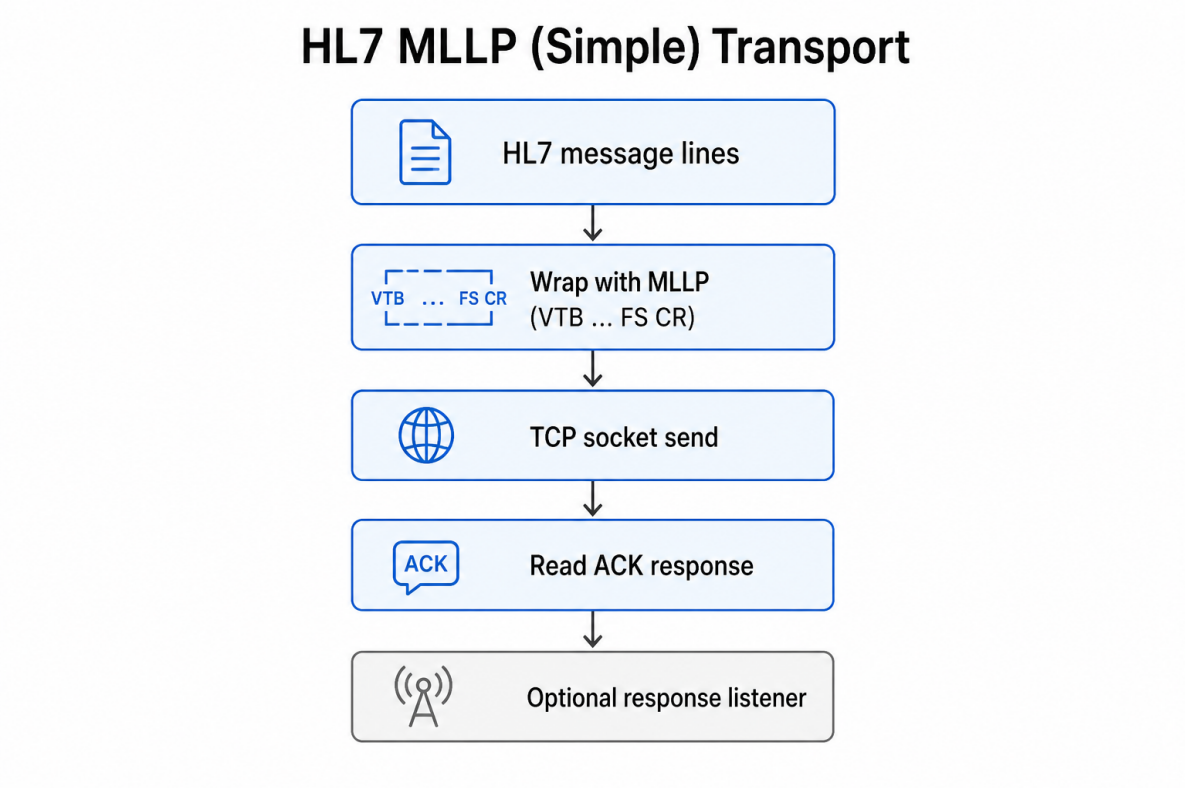


Figure 1: At a glance (plain English)

- **Wraps** each line of the transaction body in MLLP bytes (`VT` … `FS CR`).
- **Connects** to configured host/port with optional connect and read timeouts.
- **Reads** ACK into a cache unless **Ignore ACKs** is enabled.
- **Replaces** transaction stream with ACK content; may trigger a **Response Listener**.

## Purpose

This document is a **deep dive** into the **HL7 MLLP (Simple)** transport as shipped in `eip.war.hs.26R1.11`: socket lifecycle, MLLP encoding, ACK consumption, connection pooling by host:port, and idle cleanup.

| Provenance | Value |
|---|---|
| WAR | `eip.war.hs.26R1.11` |
| Module JAR | `xcs-format-hl7-1.0.1.jar` |
| Decompiler | CFR 0.152 |
| Implementation class | `com.pilotfish.eip.modules.hl7.HL7SimpleTra` |
| Decompiled path | `war-pipeline/decompiled/eip-26R1.11/.../HL` |
| Transport type string | `HL7 MLLP (Simple)` |
| Description | `Sends HL7 messages over simplified MLLP (no validation)` |

---

## 1. Conceptual model

"Simple" means **no HL7 validation** — raw line-oriented text is framed and sent. Suitable when upstream stages already produced valid HL7 or when testing against permissive receivers.

```
Transaction body (line-oriented HL7 text)
        |
        v
 TCP connect (new or pooled)
        |
        v
 Write 0x0B (VT)
 For each line: bytes + CR (0x0D)
 Write 0x1C (FS) + CR
        |
        v
 [IgnoreAck false?] read until FS+CR ACK framing
        |
        v
 responseCache -> data.setDataStream
 [ResponseListener set?] trigger listener
```

Persistent connections keyed by `host + port` string concatenation (not separated by colon in map key).

---

## 2. High-level behavior (`executeTransport`)

1. Read config: host, port, timeout, keepConnection, idleTimeout, responseListener, responseTimeout, ignoreAck.
2. `getConnection(...)` — pooled or fresh socket (section 3).
3. Obtain streams; on failure close socket and throw.
4. Allocate `responseCache`.
5. **Send loop:**
   - Write `0x0B` (VT / MLLP start).
   - `BufferedReader` reads **lines** from transaction stream; each line written as bytes + `0x0D` (CR).
   - Write `0x1C` (FS) + `0x0D` (CR) end block.
6. **ACK read** (unless Ignore ACKs):
   - Read bytes until sequence `FS (28)` then `CR (13)` terminates frame.
   - Bytes between start and end (excluding framing) copied to cache (skips lone `28` handling per decompile logic).
7. On error with keepConnection: remove connection from pool and close socket.
8. If not keepConnection: close socket after send.
9. If **Response Listener** non-empty: `TriggerableListenerUtils.trigger`.
10. `data.setDataStream(responseCache.getInputStream())`.

---

## 3. Connection management (`getConnection`)

| Mode | Behavior |
|---|---|
| keepConnection = false | Always new socket via `getSocket` |
| keepConnection = true | Reuse `connectionsByHostIp.get(host+port)` if open |
| idleTimeout > 0 | Close pooled connections inactive longer than threshold (scans all entries each call) |

`getSocket`:

- `Socket.setKeepAlive(true)`
- `connect(InetSocketAddress, timeout)` — connect timeout ms from **Connection Timeout** (`Timeout` tag)
- If **Response Timeout** > 0 → `setSoTimeout(responseTimeout)` for ACK read

`systemShutdown()` closes all pooled sockets and clears maps.

---

## 4. Configuration reference

| UI label | Tag | Type | Default | Nuances |
|---|---|---|---|---|
| Host | `Host` | String | `""` | Target hostname |

| UI label | Tag | Type | Default | Nuances |
|---|---|---|---|---|
| Port | `Port` | Integer | `-1` | 0 .. MAX_VALUE |
| Connection Timeout | `Timeout` | Integer | `-1` | Connect wait ms |
| Keep Connection | `KeepConnection` | Boolean | `false` | Persistent socket pool |
| Idle Timeout (ms) | `IdleTimeout` | Integer | `-1` | Visible when Keep Connection true; `-1` disables idle sweep |
| Response Listener | `ResponseListener` | Listener | `""` | Triggered with ACK stream |
| Response Timeout (ms) | `ResponseTimeout` | Integer | `-1` | SO_TIMEOUT for ACK read; exception on timeout |
| Ignore ACKs | `IgnoreAck` | Boolean | `false` | Advanced tab; send-only mode |

**Dependency panels**

- `KeepConnection=true` → shows **Idle Timeout**.

### 4.1 Inherited AbstractTransport

Standard send/complete/error wrapper via `executeTransport`.

---

## 5. Transaction attributes

No attributes registered on `HL7SimpleTransport`. ACK content becomes the new transaction body stream.

---

## 6. Worked example — persistent sender

- Host: `mllp.partner.local`, Port: `2575`
- Keep Connection: **true**, Idle Timeout: `300000` (5 min)
- Ignore ACKs: **false**, Response Timeout: `30000`

First message opens socket; subsequent messages reuse until idle sweep closes it. ACK HL7 populates transaction stream for downstream logging processor.

---

## 7. Known quirks and caveats

1. **Line-based sending** — segment boundaries must be represented as newlines in stream; embedded `\r` inside lines not split further.
2. **No HL7 validation** — malformed messages are sent as-is.
3. **Map key** — `host + port` string with no separator (`host1` + `234` vs `host` + `1234` collision risk is configuration-dependent).
4. **ACK parsing** — simplified byte loop; unusual framing may confuse reader.
5. **Empty ACK with IgnoreAck** — response stream may be empty cache.
6. **Error message typo** — cache failure mentions "HL7 HTTP response" in exception text (legacy string).
7. **Character encoding** — line bytes use platform default `String.getBytes()` without explicit charset.

---

## 8. Operational recommendations

| Goal | Guidance |
|---|---|
| Production MLLP | Prefer validated HL7 transports when schema compliance matters |
| High volume | Enable **Keep Connection** + idle timeout aligned with partner policy |
| Fire-and-forget | **Ignore ACKs** when partner does not require synchronous ack processing |
| ACK processing | Chain **Response Listener** to dedicated ack route |
| Timeouts | Set **Response Timeout** below partner SLA to fail fast |

---

## 9. Source index (this WAR)

| Topic | Path under `war-pipeline/decompiled/eip-26R1.11/` |
|---|---|
| HL7 MLLP (Simple) transport | `com/pilotfish/eip/modules/hl7/HL7SimpleTransport.java` |
| Abstract transport | `com/pilotfish/eip/extend/AbstractTransport.java` |
| Module JAR | `war-pipeline/jars/eip-26R1.11/xcs-format-hl7-1.0.1.jar` |

---

## 10. Pipeline provenance

```
PilotFish WARs/eip.war.hs.26R1.11
  -> xcs-format-hl7-1.0.1.jar
  -> CFR decompile
  -> Documents/Transports/26R1.11/PilotFish-HL7-MLLP-Simple-Transport-Reference-26R1.11.(md|pd
```

*Deep-dive documentation from WAR decompile – HL7 MLLP (Simple) Transport – 26R1.11 – August 2026.*